Technická univerzita v Košiciach

Fakulta elektrotechniky a informatiky

SemEval-2025 Task 9: The Food Hazard Detection Challenge

Spracovanie prirodzeného jazyka

Lukáš Vaľo
Viktor Uhrynchuk
Rostyslav Rodzhuk
Oleksiy Orlyk

2026

1 Formulácia, motivácia a ciele projektu

SemEval-2025 Task 9: The Food Hazard Detection Challenge je úloha, ktorá spočíva v automatickej identifikácii hrozieb v potravinách na základe textových údajov. Cieľom je spracovať záznamy alebo textové údaje za účelom nájdenia potenciálneho rizika, kontaminácie alebo iné problémy súvisiace s bezpečnosťou potravín.

Celkový projekt pozostáva z dvoch podúloh:

1. **Klasifikácia kategórií** -
   - `hazard-category` - kategória nebezpečenstva, napríklad alergény.
   - `product-category` - kategória výrobku, napríklad mliečny výrobok alebo mäso.
2. **Konkrétna klasifikácia označení** -
   - `hazard` - konkrétne nebezpečenstvo.
   - `product` - konkrétny typ výrobku.

V kontexte spracovania prirodzeného jazyka môžeme tento problém chápať ako automatickú klasifikáciu textu do vopred pripravených tried nebezpečenstva. Na základe datasetu, ktorý sa na takú klasifikáciu použije, je aj možné viacnásobné priraďovanie tried jednému textu, rozpoznávanie entít alebo získanie dôležitých informácií.

Základným vstupom do modelu je záznam z datasetu reprezentovaný ako opis výrobku, sťažnosť spotrebiteľa alebo hlásenie o riziku zdravia. Výstupom je predpovedaná trieda nebezpečenstva, prípadne viacero tried.

Ilustračný príklad vstupu a výstupu:

- **VSTUP:** "V balení kuracích pŕs od značky Mäsokombinát bol v posledných týždňoch zistený výskyt baktérie Salmonella."
- **VÝSTUP:** Možné nebezpečenstvo, Salmonella

1.1 Motivácia

Jednou z významných oblastí verejného zdravia je bezpečnosť zdravia. Rôzne články, príspevky, informácie o nebezpečných výrobkoch sa nachádzajú v správach, na sociálnych sieťach alebo oznamoch verejných orgánov. Ich manúalna analýza môže byť časovo náročná a pri veľkom objeme aj ťažko škálovateľná. Z takéhoto dôvodu je rozumné vytvoriť automatizovaný proces, ktorý ma slúžiť ako pomocný nástroj na identifikáciu potenciálnych rizík, nie ako náhrada odborného posudku.

Takýto automatizovaný proces môže:

- **Podporiť odborníkov**
- **Spracovať veľké množstvo dá**t
- **Preskúmať možnosti NLP metód**
- **Zrýchliť identifikáciu rizík**

1.2 Cieľ projektu

Z vyššie uvedených informácií je značné, že našim hlavným cieľom projektu je navrhnúť, implementovať a vyhodnotiť model na automatickú detekciu nebezpečenstiev súvisiacich s potravinami z textových údajov. Okrem hlavného cieľa je našou úlohou aj:

1. Analyzovať už existujúce metódy na detekciu nebezpečenstiev.
2. Preskúmať dataset, kategórie, kvalitu, jeho štruktúru a rozdelenie tried.
3. Navrhnúť baseline (základné) riešenie.
4. Navhrnúť vlastné rožširenie baseline riešenia, ako napríklad využitie predtrénovaného modelu.
5. Porovnať výsledky jednotlivých metód a analyzovať ich silné a slabé stránky

2 Prehľad dátových zdrojov, existujúcich riešení a relevantnej literatúry

Hlavným dátovým zdrojov je oficiálny dataset úlohy SemEval-2025 Task 9. Obsahuje 6644 manuálne anotovaných záznamov o incidentoch súvisiacich s potravinami získané z webových zdrojov a oznámení o stiahnutí výrobku z trhu. Samotné texty sú v angličtne a pomerne krátke.

Oficiálna stránka úlohy uvádza následujúce rozdelenie dát:

Množina	**Počet záznamov**	**Účel**
Trénovacia	5 082	Tréning modelu
Validačná	565	Výber modelu a ladenie parametrov
Testovacia	997	Záverečné vyhodnotenie

2.1 Existujúce riešenia

2.1.1 VerbaNexAI – Random Forest + TF-IDF

Autori textu použili Random Forest ako klasifikátor, reprezentáciu TF-IDF a n-gramy s dĺžkou 2 až 5 znakov. N-gramy slúžia na zachytenie časti slov, varianty zápisu a jazykové vzory. Vyhodnotenie jednotlivých modelov je možné vidieť v nasledujúcej tabuľke (Tovar et al., n.d.):

Podúloha	**Macro-F1**
Kategória nebezpečenstva a výrobku	0,5165 (51,65 %)
Konkrétne nebezpečenstvo a výrobok	0,3223 (32,23 %)

2.1.2 BERT, ModernBERT a porovnania

Narozdiel od predošlých autorov, ktorý vytvorili model na detekciu nebezpečenstiev, autori tohto textu sa zamerali na porovnanie jednotlivých modelov ako BERT,

ModernBERT v kombinácii s LinearSVC, Random Forest a LightGBM (Wang and Zhang, n.d.).

Model	**Macro-F1**
BERT	0,669
ModernBERT	0,702
ModernBERT + LinearSVC	0,808
Random Forest	0,507
LightGBM	0,498
LinearSVC	0,639

2.1.3 PuerAI – ModernBERT

Autori v tomto texte experimentovali s už predtrénovanými modelmi a uviedli, že ModernBERT je najúspešnejší v porovnaní s ostatnými s hodnotami na validačnej množine rovnými 0,7952 (79,52 %) a testovacej rovnej 0,7729 (77,29 %) (Dao et al., n.d.).

2.2 Baseline riešenie

Na základe vyššie uvedených poznatkov navrhujeme riešenie, ktoré bude jednoduché a reprodukovateľné. Takýto prístup ukáže, či ma zložitejší model vôbec zmysel. Baseline model tejto úlohy má byť použitie LinearSVC v kombinácii s reprezentáciou textu pomocou TF-IDF. V článku od ASD bola hodnota Macro-F1 na validačnej množine rovná 0,639. Navrhovaný postup implementácie:

1. Načítanie tréningových a validačných dát.
2. Zvolenie textových vstupov a cieľových štítkov hazard-category.
3. Vytvoriť reprezentáciu textov pomocou TF-IDF.

4. Natrénovať LinearSVC.

5. Vypočítať predikcie na validačných dátach.

6. Vyhodnotiť výsledky pomocou Macro-F1 a iných metrík.

3 Analýza metrík a experimentálnej metodiky

Analyzovaný notebook zodpovedá prvému z troch baseline riešení poskytnutých organizátormi (TF-IDF + logistická regresia; ďalšie dve sú BERT a CICLe) (Randl et al., 2025). Táto sekcia hodnotí, či zvolené metriky a spôsob experimentu zodpovedajú charakteru úlohy.

3.1 Použité metriky

Notebook počíta tri ukazovatele pomocou `sklearn.metrics.f1_score`:

- **Macro-F1** – nevážený priemer F1 cez triedy, $F_1^{\text{macro}} = \frac{1}{n}\sum_{c=1}^{n} \frac{2P_cR_c}{P_c+R_c}$, kde P_c a R_c sú presnosť a úplnosť triedy c (Randl et al., 2025; scikit-learn developers, 2026c),
- **Micro-F1** – F1 vypočítané z celkových počtov TP, FP a FN cez všetky vzorky (scikit-learn developers, 2026c),
- **kombinované skóre** `compute_score` – zhodné s oficiálnou metrikou súťaže (Task organizers, 2025; Randl et al., 2025):

$$S = \frac{F_1^{\text{macro}}(\mathbf{y}^h, \hat{\mathbf{y}}^h) + F_1^{\text{macro}}(\mathbf{y}^{p|h}, \hat{\mathbf{y}}^{p|h})}{2}, \tag{3.1}$$

  kde $\mathbf{y}^{p|h}$ obsahuje produkty iba tých vzoriek, pri ktorých bolo nebezpečenstvo (`hazard`) predpovedané správne.

Skóre S sa počíta osobitne pre podúlohu 1 (`hazard-category`, `product-category`) a podúlohu 2 (`hazard`, `product`).

3.2 Vhodnosť metrík

Macro-F1 dáva každej triede rovnakú váhu (scikit-learn developers, 2026c), čo je zmysluplné pri silne nevyváženom rozdelení tried (Task organizers, 2025); použitie

rovnakej metriky ako v súťaži zároveň umožňuje porovnanie s publikovanými výsledkami. Slabinou je nestabilita pri veľkom počte tried. Podľa zdrojov má `hazard` 128 až 261 a `product` 1 142 až 1 256 hodnôt (Task organizers, 2025; Randl et al., 2025). Vývojová množina má približne 1 000 vzoriek (20 % z 5 082 (Task organizers, 2025)), čo je v priemere 4 až 8 vzoriek na triedu `hazard` a menej ako jedna vzorka na triedu `product`. Pri triede s jediným príkladom pritom jedna chyba zmení jej F1 z 1 na 0. Scikit-learn navyše štandardne priemeruje cez zjednotenie tried z `y_true` a `y_pred` (scikit-learn developers, 2026c), takže predikcia triedy, ktorá sa v referenčných štítkoch nevyskytuje, má F1 rovné 0 a znižuje skóre.

Micro-F1 je pri klasifikácii s jedným štítkom na vzorku zhodná s presnosťou (accuracy), pretože každá chyba je práve jedným FP a jedným FN, a dominujú jej frekventované triedy. Rozdiel micro − macro je v Tab. 4 – 2 0,20 až 0,40, čo naznačuje horšie výsledky na zriedkavejších triedach, hoci notebook po triedach nevyhodnocuje. Micro-F1 nie je súčasťou oficiálneho hodnotenia (Randl et al., 2025), preto má iba doplnkovú úlohu.

Kombinované skóre S zohľadňuje prioritu nebezpečenstva: pri chybnom `hazard` je $S = 0$ a pri bezchybnom `hazard` a úplne chybnom `product` je $S = 0{,}5$ (Randl et al., 2025). Produkt sa hodnotí len na podmnožine správne určených nebezpečenstiev, ktorá je pre každý model iná, takže $F_1^{p|h}$ dvoch modelov nie je počítané na rovnakých vzorkách. Zo zverejnených hodnôt (S a macro-F1 pre `hazard` z Tab. 4 – 2) vychádza $F_1^{p|h} = 2S - F_1^{\text{macro}}(\mathbf{y}^h, \hat{\mathbf{y}}^h)$ približne 0,44 pre podúlohu 1 a 0,10 pre podúlohu 2, s presnosťou $\pm 0{,}01$ danou zaokrúhlením.

Chýbajúce prvky. Notebook importuje `classification_report` a `ConfusionMatrixDisplay`, ale nepoužíva ich, takže chýba analýza po triedach aj matica zámen. Parameter `zero_division=0` je nastavený len pri výpise v cykle; v `compute_score` sa použije predvolená hodnota `'warn'`, ktorá sa správa ako 0, no pri nedefinovanom F1 vyvolá varovanie (scikit-learn developers, 2026c). Skóre sa uvádza ako jedno číslo bez intervalu spoľahlivosti.

3.3 Experimentálna metodika

Vstup: iba stĺpec `title`; `text`, dátum a krajina sa nepoužívajú. Titulky majú v priemere 88 znakov, plné texty 2 329 (Randl et al., 2025). Väčšina systémov v súťaži používala `title` aj `text` a ich kombinácia podľa autorov súťaže zlepšovala výsledky, pričom počet systémov len s `title` je príliš malý na definitívny záver (Randl et al., 2025).

Delenie dát: `train_test_split(test_size=0.2, random_state=2024)` z `incidents_train.csv`; komentár v notebooku sa odvoláva na (Randl et al., 2024) (v notebooku je uvedený rok 2023, odkaz smeruje na Findings ACL 2024). Delenie nie je stratifikované, zatiaľ čo oficiálne rozdelenie 5 082/565/997 bolo stratifikované podľa `hazard` (Randl et al., 2025), takže rozdelenie tried vo vývojovej množine nemusí zodpovedať validačnej a testovacej množine.

Využitie oficiálnych množín: vyhodnotenie prebieha iba na 20 % trénovacích dát. Označená validačná množina sa v notebooku nenačítava. Označená testovacia množina (podľa komentára v notebooku) sa načíta, ale používa sa len na predikciu a export.

Duplicity: autori súťaže uvádzajú 42 duplicitných záznamov, z toho 6 spoločných pre trénovaciu a validačnú množinu a 7 pre trénovaciu a testovaciu množinu (menej ako 1 % dát) (Randl et al., 2025). Notebook duplicity neodstraňuje; zdroj neuvádza, či sa duplicity vyskytujú aj vo vnútri trénovacej množiny.

Trénovanie: štyri nezávislé modely (po jednom pre každý štítok) s rovnakou konfiguráciou, bez ladenia hyperparametrov, bez váhovania tried a bez opakovaných behov. Systémy riešiace úlohy samostatne dosahovali v súťaži lepšie výsledky ako systémy so zdieľaným modelom (Randl et al., 2025), no nezávislé modely nezaručujú hierarchickú konzistenciu (predpovedané `hazard` nemusí patriť do predpovedanej `hazard-category`).

Predikcia na testovacej množine: model je natrénovaný len na 80 % trénovacích dát; v oficiálnej fáze hodnotenia bolo povolené trénovať aj na validačnej množine (Randl et al., 2025).

Skóre S z notebooku (vývojová množina) nie je priamo porovnateľné s hodnotami baseline v článku (Tab. 3 – 1), pretože sa počíta na inej množine. Článok pri baseline hodnotách výslovne neuvádza, na ktorej množine boli vypočítané; porovnanie s vlastnou reprodukciou je v kap. 5.

Tabuľka 3 – 1 Skóre S: notebook (vývojová množina) a baseline TF-IDF + LR podľa článku.

	Podúloha 1	Podúloha 2
Notebook, 20 % trénovacích dát	0,449	0,121
Baseline TF-IDF + LR podľa (Randl et al., 2025)	0,498	0,183

4 Oficiálne baseline riešenie: implementácia a reprodukovateľnosť

4.1 Postup riešenia

1. Načítanie `data/incidents_train.csv` a náhodné rozdelenie na trénovaciu (80 %) a vývojovú (20 %) množinu.
2. Načítanie `data/incidents_test.csv`; podľa komentára v notebooku ide o označenú testovaciu množinu.
3. Zostavenie `Pipeline`: TF-IDF vektorizácia a logistická regresia.
4. Pre každý zo štyroch štítkov: `fit` na trénovacej množine, predikcia na vývojovej aj testovacej množine; predikcie testovacej množiny prepíšu v `testset` stĺpce s referenčnými štítkami.

5. Výpočet macro-F1, micro-F1 a skóre S na vývojovej množine.

6. Uloženie predikcií (štyri stĺpce a index záznamu) do `submission.csv` a zabalenie do `submission.zip`, ktorý je podľa komentára v notebooku určený na nahratie na Codalab.

4.2 Model

Vstupný text sa reprezentuje znakovými n-gramami 2–5 (rovnaká reprezentácia ako pri (Tovar et al., n.d.)) a klasifikátorom je logistická regresia. Voľbu logistickej regresie nad TF-IDF podporuje zistenie, že tento model prekonáva RoBERTa a XLM-R pri triedach s nízkou podporou (Randl et al., 2024). Vektorizátor je súčasťou `Pipeline`, takže slovník a IDF sa učia len z trénovacích dát (scikit-learn developers, 2026a).

Tabuľka 4 – 1 Parametre pipeline (scikit-learn developers, 2026a,b).

Parameter	Význam
`analyzer='char'`, `ngram_range=(2,5)`	znakové n-gramy dĺžky 2 až 5
`strip_accents='unicode'`	odstránenie akcentov (normalizácia NFKD)
`max_df=0.5`	vyradenie n-gramov s podielom dokumentov > 50 %
`min_df=5`	vyradenie n-gramov v menej ako 5 dokumentoch
`LogisticRegression (max_iter=1000)`	ostatné parametre predvolené: L2 penalizácia, $C = 1$, solver `lbfgs`, bez `class_weight`

Tá istá `Pipeline` sa v cykle cez štyri štítky opakovane trénuje od začiatku (`fit`)

len na stĺpci `title` trénovacej množiny; po každom tréningu sa predikcie ukladajú do vývojovej aj testovacej množiny.

4.3 Výsledky

Tabuľka 4 – 2 Výsledky notebooku na vývojovej množine (hodnoty z výstupu notebooku).

Štítok	Macro-F1	Micro-F1
`hazard-category`	0,46	0,81
`product-category`	0,39	0,66
Skóre S, podúloha 1: 0,449		
`hazard`	0,14	0,54
`product`	0,07	0,27
Skóre S, podúloha 2: 0,121		

4.4 Reprodukovateľnosť

Silné stránky:

- pevný `random_state=2024` pri delení a explicitne zadané hyperparametre vektorizátora,
- solver `lbfgs` nepoužíva generátor náhodných čísel (parameter `random_state` sa uplatňuje len pri solveroch `sag`, `saga` a `liblinear` (scikit-learn developers, 2026b)),
- uložené výstupy pochádzajú z postupného behu buniek (`execution_count` 1–6) a hodnoty macro-F1, micro-F1 aj S sú identické v dvoch uložených behoch notebooku, ktorých exportná cesta ukazuje na Linux, resp. Windows,
- žiadne externé modely ani sieťové volania.

Nedostatky:

- notebook neuvádza verzie knižníc ani spôsob inštalácie (metadáta uvádzajú kernel `nlp_two` a Python 3.12); dokumentácia scikit-learn pripúšťa, že koeficienty modelu sa môžu mierne líšiť medzi strojmi a verziami knižnice (scikit-learn developers, 2026b),
- notebook neobsahuje príkazy na získanie dát, očakáva súbory `data/incidents_train.csv` a `data/incidents_test.csv` (relatívne cesty),
- označená testovacia množina sa nevyhodnocuje a príkaz `testset[label] = ...` prepíše jej referenčné štítky predikciami, takže po behu cyklu ich v pamäti nemožno použiť na výpočet skóre bez opätovného načítania.

4.5 Postup spustenia

Notebook importuje knižnice `pandas` a `scikit-learn` (Pedregosa et al., 2011); moduly `os` a `shutil` sú súčasťou štandardnej knižnice. Na jeho spustenie je potrebné prostredie Jupyter (balík `jupyterlab`). Odporúča sa izolované prostredie so Pythonom 3.12, ktorý uvádzajú metadáta notebooku. Dá sa vytvoriť dvoma spôsobmi:

- **venv:** vytvoriť virtuálne prostredie nástrojom `venv`, aktivovať ho a cez `pip` nainštalovať `pandas`, `scikit-learn` a `jupyterlab`; verzie knižníc je vhodné zafixovať do súboru `requirements.txt`,
- **conda:** vytvoriť prostredie nástrojom `conda` s rovnakými tromi balíkmi (kanál `conda-forge`) a verzie zafixovať exportom do súboru `environment.yml`.

Dáta sa získajú stiahnutím repozitára úlohy (Task organizers, 2025), ktorý v priečinku `data/` obsahuje súbory `incidents_train.csv`, `incidents_valid.csv` a `incidents_test.csv`. Tento priečinok sa skopíruje vedľa notebooku, pretože cesty v notebooku sú relatívne (`data/...`). Notebook sa potom spustí v aktivovanom prostredí vykonaním všetkých buniek v poradí od začiatku (Restart kernel and Run All Cells).

5 Vlastné riešenie: TF-IDF a LinearSVC

5.1 Návrh a implementácia

Riešenie je implementované v notebooku `LinearSVC_Food_Hazard_Baseline.ipynb` podľa krokov z kap. 2.2. Oproti oficiálnemu riešeniu sa líši v troch bodoch:

- klasifikátor LinearSVC namiesto logistickej regresie pri rovnakej reprezentácii (TF-IDF nad znakovými n-gramami 2–5, vstup `title`), takže rozdiel výsledkov možno pripísať klasifikátoru; LinearSVC je postavený na knižnici liblinear a viactriedny problém rieši schémou jeden proti zvyšku (scikit-learn developers, 2026d),
- vyhodnotenie na oficiálnej označenej validačnej (565 vzoriek) a testovacej (997 vzoriek) množine; model sa trénuje na celej trénovacej množine a testovacia množina sa nepoužíva na žiadne rozhodnutie (Task organizers, 2025),
- explicitne nastavené seedy a automatické ukladanie metadát behu.

Rovnako ako v oficiálnom riešení sa pre každý zo štyroch štítkov trénuje samostatný model. V jednom experimente sa vyhodnocujú tri modely: LinearSVC (hlavný), LinearSVC s `class_weight='balanced'` (váhy tried nepriamo úmerné ich početnosti (scikit-learn developers, 2026d)) a logistická regresia v oficiálnej konfigurácii ako referencia. Hyperparametre sa neladili. Okrem macro-F1, micro-F1 a skóre S sa počítajú 95 % percentilové bootstrap intervaly skóre S (500 opakovaní, rovnaké vzorky pre všetky modely).

5.2 Reprodukovateľnosť

- **Seedy:** jediný `SEED=2024` pre `LinearSVC`, delenie dát v kontrolnom behu a bootstrap; `random_state` ovplyvňuje LinearSVC len pri `dual=True` (scikit-learn developers, 2026d). Podľa dokumentácie bola pre `dual` pridaná možnosť

Tabuľka 5 – 1 Parametre LinearSVC.

Parameter	Hodnota a dôvod
`C=1.0`	predvolená hodnota (scikit-learn developers, 2026d)
`penalty`, `loss`	predvolené: L2, `squared_hinge` (scikit-learn developers, 2026d)
`dual=True`	nastavené explicitne, pretože `random_state` sa uplatňuje len pri duálnej formulácii (scikit-learn developers, 2026d)
`max_iter=10000`	predvolená hodnota je 1 000; vyššia hodnota zabraňuje predčasnému zastaveniu optimalizácie
`random_state=2024`	jediný seed pre celý notebook

`'auto'` vo verzii 1.3 a mala sa stať predvolenou od verzie 1.5 (scikit-learn developers, 2026d), preto je hodnota nastavená explicitne.

- **Konvergencia:** v hlavnom behu nevzniklo žiadne `ConvergenceWarning` (štyri štítky, tri modely).
- **Metadáta behu:** verzie knižníc, kontrolné súčty dát (SHA-256) a parametre modelov sa ukladajú do `run_info.json`. Beh, z ktorého pochádzajú výsledky, použil Python 3.12.14 (Windows 11), `numpy` 2.5.2, `pandas` 3.0.6, `scikit-learn` 1.9.1 a `matplotlib` 3.11.2; kontrolné súčty začínajú `e1309f1d6d68` (train), `80dc04c1d9b0` (valid) a `3dd8102b0031` (test).
- **Prostredie:** rovnaké ako v podsekcii Postup spustenia (kap. 4.5), navyše s knižnicou `matplotlib`.
- **Overenie reprodukcie oficiálneho riešenia:** referenčný model v protokole oficiálneho notebooku (80/20 delenie trénovacích dát, `random_state=2024`)

dosiahol $S = 0{,}449$ a 0,121, čo je zhodné s uloženým výstupom oficiálneho notebooku.

5.3 Výsledky

Tabuľka 5 – 2 Skóre S s 95 % bootstrap intervalom.

Model	Podúloha 1	Podúloha 2
Validačná množina		
LinearSVC	0,678 [0,595; 0,726]	0,409 [0,373; 0,448]
LinearSVC, `balanced`	0,694 [0,618; 0,746]	0,355 [0,316; 0,382]
LR (oficiálna konfigurácia)	0,498 [0,454; 0,564]	0,183* [0,187; 0,234]
Testovacia množina		
LinearSVC	0,674 [0,624; 0,744]	0,375 [0,349; 0,407]
LinearSVC, `balanced`	0,657 [0,613; 0,725]	0,351 [0,320; 0,371]
LR (oficiálna konfigurácia)	0,483 [0,451; 0,561]	0,129* [0,136; 0,166]

* bodový odhad leží mimo bootstrap intervalu.

LinearSVC prekonáva referenčnú logistickú regresiu vo všetkých štyroch porovnaniach (dve podúlohy, dve množiny): rozdiel v skóre S je 0,180 (podúloha 1) a 0,226 (podúloha 2) na validačnej a 0,191 a 0,245 na testovacej množine, pričom sa 95 % intervaly neprekrývajú ani v jednom prípade. Pri logistickej regresii v podúlohe 2 leží bodový odhad mimo bootstrap intervalu na oboch množinách; príčinu sme neurčili, interval tohto modelu preto treba interpretovať opatrne. Záver sa tým nemení, keďže interval LR leží ďaleko pod intervalom LinearSVC (0,187–0,234 voči 0,373–0,448 na validačnej množine).

Variant `class_weight='balanced'` nedáva konzistentné zlepšenie. V podúlohe 1 je lepší na validačnej (0,694 voči 0,678) a horší na testovacej množine (0,657 voči 0,674) pri silne sa prekrývajúcich intervaloch. V podúlohe 2 je nižší na oboch množinách (0,355 voči 0,409 a 0,351 voči 0,375), no intervaly sa čiastočne prekrývajú.

Párové bootstrap rozdiely sa v notebooku len zobrazujú a neukladajú, preto sa tu neuvádzajú.

Tabuľka 5 – 3 Macro-F1 podľa štítku.

Model	`hazard-category`	`product-category`	`hazard`	`product`
Validačná množina				
LinearSVC	0,663	0,646	0,475	0,281
LinearSVC, `balanced`	0,679	0,727	0,410	0,232
LR (oficiálna konfigurácia)	0,515	0,465	0,200	0,122
Testovacia množina				
LinearSVC	0,648	0,700	0,428	0,275
LinearSVC, `balanced`	0,620	0,683	0,425	0,240
LR (oficiálna konfigurácia)	0,508	0,452	0,154	0,081

LinearSVC má vyššie macro-F1 než LR pri všetkých štítkoch na oboch množinách. Relatívne zlepšenie je najväčšie pri jemných štítkoch: macro-F1 pre `hazard` a `product` je 2,3- až 3,4-krát vyššie, pri kategóriách približne 1,3- až 1,5-krát. Rozdiel medzi micro a macro hodnotou pretrváva (pre `hazard-category` 0,835 voči 0,663 na validačnej a 0,861 voči 0,648 na testovacej množine), takže slabšie výsledky na zriedkavých triedach zostávajú. Absolútna úroveň na testovacej množine (0,674 a 0,375) je pod najlepšími systémami súťaže (0,8223 v podúlohe 1 a 0,5473 v podúlohe 2 (Randl et al., 2025)).

Kontrolný beh podľa protokolu oficiálneho notebooku. Rovnaký protokol (20 % trénovacích dát, `random_state=2024`) bol zopakovaný pre všetky modely (Tab. 5 – 4). Referenčný model dáva hodnoty zhodné s oficiálnym notebookom. LinearSVC dosahuje na 20 % trénovacích dát nižšie skóre než na validačnej a testovacej množine; pritom sa tu trénuje len na 80 % dát.

Zhoda s hodnotami z článku. Referenčný model dosahuje na validačnej množine $S = 0{,}498$ a 0,183, čo sa na tri desatinné miesta zhoduje s hodnotami baseline

Tabuľka 5 – 4 Skóre S v protokole oficiálneho notebooku (20 % trénovacích dát).

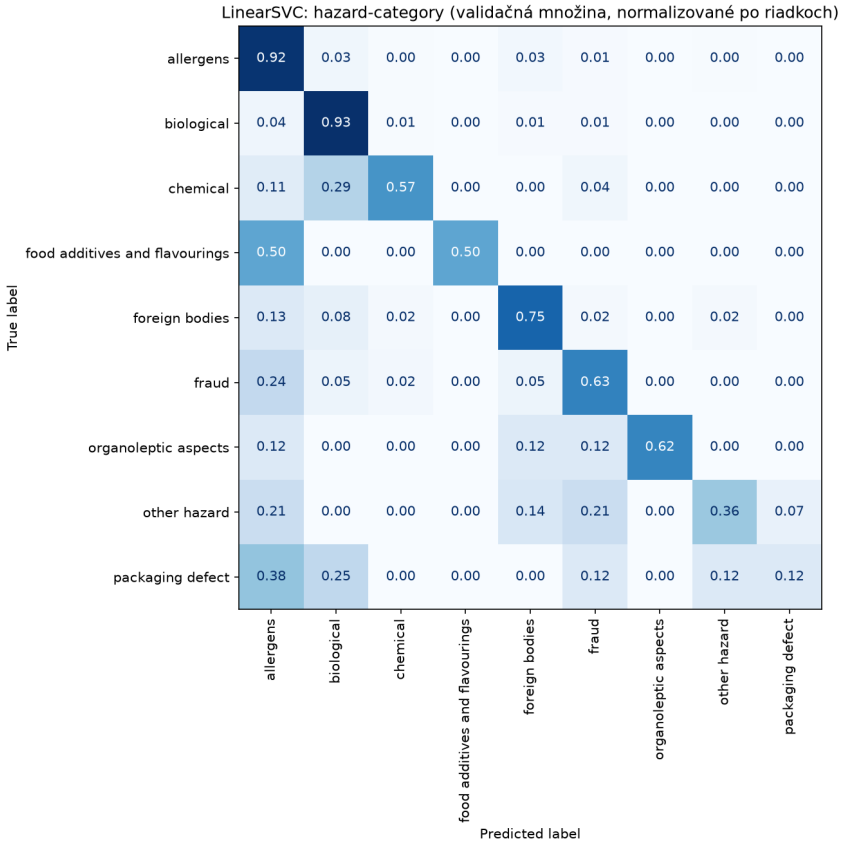
Model	Podúloha 1	Podúloha 2
LinearSVC	0,636	0,310
LinearSVC, `class_weight='balanced'`	0,595	0,287
LR (oficiálna konfigurácia)	0,449	0,121
Oficiálny notebook (uložený výstup)	0,449	0,121

TF-IDF + LR v tabuľkách článku (Randl et al., 2025); na testovacej množine vychádza 0,483 a 0,129. Článok pri baseline hodnotách výslovne neuvádza množinu, preto zhoda naznačuje, že boli vypočítané na validačnej množine. To vysvetľuje, prečo lokálny beh oficiálneho notebooku (0,449 a 0,121 na 20 % trénovacích dát) nezodpovedá hodnotám v článku: ide o iné množiny.

5.4 Analýza chýb

Matica zámen pre `hazard-category` (Obr. 5 – 1) obsahuje 9 z 10 kategórií; kategória `migration` sa nevyskytuje ani v referenčných štítkoch validačnej množiny, ani v predikciách. Najlepšie sú rozpoznané `allergens` (0,92) a `biological` (0,93), ktoré podľa poradia tried v (Randl et al., 2025) patria medzi najčastejšie. Slabšie sú `chemical` (0,57), `other hazard` (0,36) a `packaging defect` (0,12). Chyby smerujú najčastejšie do triedy `allergens` (pri siedmich z ôsmich ostatných tried, dvakrát s rovnakou hodnotou ako iná trieda); výnimkou je `chemical`, ktorá sa zamieňa s `biological` (0,29). Hodnoty ako 0,12; 0,25 a 0,38 naznačujú, že niektoré riadky matice vychádzajú z veľmi malého počtu vzoriek.

Z uložených predikcií vyplývajú dva ďalšie poznatky. Po prvé, model predikuje obmedzený počet hodnôt: 65 (valid) a 81 (test) rôznych hodnôt `hazard` a 229, resp. 327 hodnôt `product`, kým podľa zdrojov má `hazard` 128 až 261 a `product` 1 142 až 1 256 hodnôt (Task organizers, 2025; Randl et al., 2025). Po druhé, nezávislé modely nezaručujú konzistenciu štítkov. Za predpokladu, že každé nebezpečenstvo



Obrázok 5 – 1 Matica zámen LinearSVC pre `hazard-category` na validačnej množine (normalizovaná po riadkoch).

patrí práve do jednej kategórie (zdroj uvádza zaradenie pomocou ontológií (Randl et al., 2025)), sa kategória pri 54 z 565 predikcií (9,6 %) na validačnej a pri 102 z 997 (10,2 %) na testovacej množine líši od kategórie, s ktorou sa dané predpovedané `hazard` v predikciách vyskytuje najčastejšie. Ide o odhad z predikcií, nie o porovnanie s referenčným mapovaním.

5.5 Zhrnutie a obmedzenia

LinearSVC zvyšuje skóre S oproti oficiálnej konfigurácii približne o 0,18 až 0,25 pri rovnakej TF-IDF reprezentácii, pričom intervaly spoľahlivosti sa neprekrývajú; najväčší relatívny prínos je pri jemných štítkoch `hazard` a `product`. Výsledky treba čí-

ať s týmito obmedzeniami: používa sa len `title`, hyperparametre sa neladili, každý model bol vyhodnotený v jednom behu na jednej platforme, štítky sa predikujú nezávisle (bez zaručenej hierarchickej konzistencie), percentilové intervaly majú pri logistickej regresii v podúlohe 2 uvedenú anomáliu a prínos váhovania tried zostáva nepreukázaný.

6 Záver

Cieľom projektu bolo analyzovať, implementovať a experimentálne vyhodnotiť metódy automatickej detekcie nebezpečenstiev v potravinách na základe textových údajov v rámci úlohy SemEval-2025 Task 9: The Food Hazard Detection Challenge.

V úvodnej časti práce sme analyzovali dostupné dátové zdroje, existujúce riešenia a metódy klasifikácie textu. Zamerali sme sa na vhodnosť hodnotiacich metrík a experimentálnej metodiky, pričom sme zohľadnili najmä nevyvážené rozdelenie tried v datasete. Následne sme preskúmali a reprodukovali oficiálne baseline riešenie založené na reprezentácii TF-IDF a logistickej regresii.

Na základe získaných poznatkov sme implementovali vlastné riešenie využívajúce TF-IDF a klasifikátor LinearSVC. Riešenie sme vyhodnotili na oficiálnej validačnej a testovacej množine a jeho výsledky porovnali s referenčnou logistickou regresiou. Na validačnej množine dosiahol LinearSVC kombinované skóre S rovné 0,678 v prvej podúlohe a 0,409 v druhej podúlohe, zatiaľ čo referenčný model dosiahol hodnoty 0,498 a 0,183. Na testovacej množine dosiahol LinearSVC skóre 0,674 a 0,375, čím prekonal referenčné riešenie v oboch podúlohách.

Experimenty zároveň ukázali, že použitie váhovania tried v našich experimentoch neprinieslo konzistentné zlepšenie výsledkov. Analýza chýb poukázala na problémy pri rozpoznávaní menej zastúpených tried a obmedzenia nezávislej predikcie jednotlivých štítkov.

Medzi hlavné obmedzenia riešenia patrí využitie iba názvov textových záznamov, absencia ladenia hyperparametrov a nerovnomerné zastúpenie tried. Možným pokračovaním projektu je využitie celého textu záznamov, optimalizácia parametrov modelu a preskúmanie pokročilejších metód klasifikácie.

Dosiahnuté výsledky ukazujú, že kombinácia TF-IDF a LinearSVC predstavuje vhodný prístup k automatickej klasifikácii potravinových nebezpečenstiev a v rámci vykonaných experimentov dosahuje lepšie výsledky než referenčná logistická regresia.

Literatúra

Dao, J., Li, Z., Tang, X., Su, Y., Zhou, Q., He, W. and Lan, X. (n.d.). Puerai at semeval-2025 task 9: Research on food safety data classification using modernbert, *Technical report*.
URL: *https://github.com/*

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011). Scikit-learn: Machine learning in python, *Journal of Machine Learning Research* **12**: 2825–2830.
URL: *https://jmlr.org/papers/v12/pedregosa11a.html*

Randl, K., Pavlopoulos, J., Henriksson, A. and Lindgren, T. (2024). Cicle: Conformal in-context learning for largescale multi-class food risk classification, *Findings of the Association for Computational Linguistics: ACL 2024*, Association for Computational Linguistics, Bangkok, Thailand, pp. 7695–7715.
URL: *https://aclanthology.org/2024.findings-acl.459*

Randl, K., Pavlopoulos, J., Henriksson, A., Lindgren, T. and Bakagianni, J. (2025). Semeval-2025 task 9: The food hazard detection challenge, *Proceedings of the 19th International Workshop on Semantic Evaluation (SemEval-2025)*, Association for Computational Linguistics, Vienna, Austria, pp. 2523–2534.
URL: *https://aclanthology.org/2025.semeval-1.325/*

scikit-learn developers (2026a). sklearn.feature_extraction.text.tfidfvectorizer – scikit-learn documentation.
URL: *https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html*

scikit-learn developers (2026b). sklearn.linear_model.logisticregression – scikit-learn

documentation.
URL: *https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html*

scikit-learn developers (2026c). sklearn.metrics.f1_score – scikit-learn documentation.
URL: *https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html*

scikit-learn developers (2026d). sklearn.svm.linearsvc – scikit-learn documentation.
URL: *https://scikit-learn.org/stable/modules/generated/sklearn.svm.LinearSVC.html*

Task organizers (2025). Semeval 2025 task 9: The food hazard detection challenge, Webová stránka úlohy a GitHub repozitár (dáta, ukážkový kód, pravidlá hodnotenia).
URL: *https://food-hazard-detection-semeval-2025.github.io*

Tovar, A. M., Puertas, E. and Martinez-Santos, J. C. (n.d.). Automatic detection of food hazards, *Technical report*.
URL: *https://github.com/VerbaNexAI/SemEval2025*

Wang, W. and Zhang, W. (n.d.). Cicl at semeval-2025 task 9: A pilot study on different machine learning models for food hazard detection challenge, *Technical report*.
URL: *https://dzlab.github.io/dltips/en/pytorch/*